DOCUMENTO DE TRABAJO · EQUIPO DE DESARROLLO

PLAN MAESTRO DE CUMPLIMIENTO

INGENIERÍA DE DATOS II

Auditoría del modelado, diseño de la solución, implementación y pruebas para cumplir los **40 requisitos de promoción**.

04
MOTORES DE DATOS

40
REQUISITOS

01
PLAN INTEGRADO

POSTGRESQL / MONGODB / CASSANDRA / NEO4J

Objetivo: disponer de una plataforma demostrable, con datos de prueba, procesos operativos y evidencia de aceptación por cada punto solicitado por el profesor.

GRUPO 4

Versión 1.0 · 23 de octubre de 2026

01 / PUNTO DE PARTIDA

Qué debemos resolver

Documento operativo para el equipo. Una estructura de datos bien diseñada no equivale a una funcionalidad probada.

40
requisitos del profesor

04
bases especializadas

00
requisitos verificados en ejecución

Regla de aceptación

> **Condición indicada por el profesor:** cada punto debe estar presente, operativo y tener datos para devolver un resultado. El proyecto no puede considerarse aprobado solo porque existe un DER o un archivo CQL.

Diagnóstico del material recibido

- **PostgreSQL · base transaccional** El DER incluye usuarios, roles, entidades logísticas, facturas, pagos y cuentas corrientes. Faltan reglas configurables, cierre mensual, conceptos facturables y solicitudes de reportes.
- **Cassandra · telemetría** Hay cinco tablas por patrones de consulta y un flujo Q01–Q12. Falta validar CQL, ingesta, consultas entre días y actualización segura de los registros “últimos”.
- **Neo4j · red de relaciones** Incluye ocho etiquetas y sus relaciones principales. Deben unificarse identificadores con PostgreSQL y corregirse la cardinalidad de envío a contenedor.
- **MongoDB · documentos flexibles** La primera entrega define su responsabilidad para eventos, alertas, mensajes, reportes y reclamos. No se adjuntó un modelo físico de colecciones para verificar.

> **Alcance de esta guía:** el documento propone un diseño y una ruta de implementación, pero no certifica que exista código ejecutable. Cualquier campo, colección o proceso nuevo aparece como propuesta técnica.

Arquitectura políglota

Una única plataforma con responsabilidades explícitas para cada motor y mecanismos de coordinación entre datos.

PRESENTACIÓN / API DE NEGOCIO

ABM · monitoreo · facturación · reportes · mensajes · mapas · predicciones

FUENTE TRANSACCIONAL

POSTGRESQL

Maestros, relaciones, reglas, incidentes, contabilidad y solicitudes.

SERIES TEMPORALES

CASSANDRA

Telemetría histórica, lecturas y posiciones GPS por sensor/contenedor.

DOCUMENTOS

MONGODB

Eventos logísticos, alertas, mensajes y resultados flexibles.

RECORRIDOS Y GRAFOS

NEO4J

Rutas, vínculos, tramos y relaciones entre actores y envíos.

Contratos entre motores

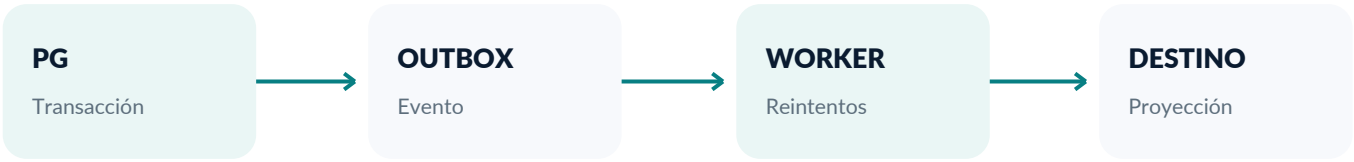
- **Identidad compartida** El UUID de PostgreSQL identifica la misma entidad en Cassandra, MongoDB y Neo4j; en Neo4j se almacena como String.
- **Origen de verdad** Las entidades comerciales y financieras se crean y modifican en PostgreSQL. Los otros motores no deben convertirse en fuentes competidoras.
- **Actualización coordinada** Recomendar eventos de dominio/outbox con consumidores idempotentes; para el TP puede implementarse un worker simple, pero deben probarse reintentos.
- **Consistencia de lectura** La API documenta qué datos son inmediatos y cuáles pueden demorar en sincronizarse; reportes financieros se calculan sobre PostgreSQL.

Decisiones que evitan errores

Estas condiciones cruzan todos los módulos; deben acordarse antes de empezar a programar funcionalidades aisladas.

ID	Decisión	Criterio obligatorio de diseño
D01	IDs y fechas	UUID compartido, timestamps con zona/UTC y fechas de negocio definidas; clave natural solo si está justificada.
D02	Trazabilidad	Cada operación tiene id de correlación, creador, timestamp y registro de error si falla.
D03	Integridad	Claves foráneas y CHECK en PostgreSQL; validaciones equivalentes en las escrituras de otros motores.
D04	Sin duplicados	Facturación automática, alertas y escrituras de telemetría con clave idempotente o identificador de evento.
D05	Historia temporal	Asignaciones envío-contenedor y sensor-contenedor con inicio/fin para consultar el pasado sin perder contexto.
D06	Prueba reproducible	Seed de datos + script de carga + consultas/requests + salida esperada + capturas o exportación.

Flujo mínimo de coordinación



Recomendación de implementación: no se necesitan transacciones distribuidas entre los cuatro motores. Guardar el evento junto con la transacción principal y procesarlo con reintentos evita pérdidas cuando un destino está caído.

Reglas de negocio que hay que escribir explícitamente

- **Estados** Definir estados permitidos de envíos, facturas, pagos, incidentes y alertas; establecer transiciones válidas.
- **País de los reportes** Especificar si el filtro/agrupación utiliza país de origen, país de destino o país del incidente.
- **Métricas** Definir atrasado, tiempo de entrega, volumen transportado y promedio de incidentes, incluyendo filtros y denominadores.
- **Datos sensibles** Roles y autorizaciones en cada endpoint; un cliente no debe poder consultar facturas, envíos ni mensajes ajenos.

Correcciones del modelo I

Preservar el DER actual. Agregar estructuras solo donde haya una funcionalidad exigida sin soporte suficiente.

P01

regla_alerta

Campos recomendados: id_regla, id_cliente, id_sensor?, tipo_regla, operador, umbral, severidad, activa, fecha_creacion

Justificación: Permite ABM de reglas y condiciones de disparo por cliente/sensor.

COBERTURA R09, R23, R26, R32

P02

incidente.id_tramo

Campos recomendados: FK opcional a tramo + índice (id_tramo, fecha)

Justificación: Atribuye incidentes a tramos/rutas sin obligar a que todos pertenezcan a un tramo.

COBERTURA R25, R27, R37

P03

asignacion_sensor_contenedor

Campos recomendados: id_asignacion, id_sensor, id_contenedor, fecha_inicio, fecha_fin

Justificación: Conserva histórico si el negocio permite mover sensores entre contenedores.

COBERTURA R08, R19, R22

P04

sesion

Campos recomendados: Agregar fecha_expiracion, fecha_revocacion y/o token_hash si se manejan sesiones propias

Justificación: Permite detectar sesiones vigentes y revocar accesos, no solo guardar inicio/fin.

COBERTURA R03

> **No asumir cambios innecesarios:** si se acuerda contractualmente que un sensor nunca cambia de contenedor, P03 puede omitirse. En cambio, el control de asignaciones históricas sí debe resolverse en algún lugar.

Correcciones del modelo II

La operación financiera debe dejar trazabilidad, impedir duplicados y permitir conciliación de saldos.

P05 factura_detalle

id_detalle, id_factura, concepto, cantidad, precio_unitario, subtotal. Registrar transporte, servicios y reportes facturables.

P06 tarifa_servicio

id_tarifa, concepto, criterio_calculo, precio, vigencia_desde, vigencia_hasta, moneda. Reglas usadas en facturación automática.

P07 cierre_cuenta

id_cierre, id_cuenta, periodo, saldo_inicial, total_debitos, total_creditos, saldo_final, fecha_cierre, estado. UNIQUE(id_cuenta, periodo).

P08 solicitud_reporte

id_solicitud, id_cliente, id_usuario, tipo_reporte, parametros JSONB, estado, precio, fecha_solicitud, id_factura?

P09 titular financiero

CHECK: exactamente uno de id_cliente, id_proveedor o id_operador en cuenta_corriente y factura, o rediseño consistente del titular. Identificar emisor/receptor y naturaleza.

Invariantes de negocio a probar

- **Conciliación** Saldo de cuenta = saldo inicial + débitos - créditos, según convención definida. El saldo no puede divergir de movimientos válidos.
- **Imputación** Cada pago debe imputarse de manera verificable a una factura o saldo, incluyendo pagos parciales si se permiten.
- **Cierre** Un período cerrado no admite cambios retroactivos sin reapertura controlada o ajuste registrado.
- **Automatización** Dos ejecuciones del mismo lote no deben emitir facturas ni cargos duplicados.

Modelo de telemetría

El draw.io recibido contiene cinco tablas y un flujo Q01–Q12. Mantener el enfoque orientado a consultas.

Tabla	PK / orden del clustering	Finalidad
lecturas_por_sensor_dia	(id_sensor, dia) · fecha_hora DESC, id_lectura ASC	Historial por sensor; análisis de temperatura, batería, tendencias.
lecturas_por_contenedor_dia	(id_contenedor, dia) · fecha_hora DESC, id_sensor ASC, id_lectura ASC	Historial combinado de sensores asociados a contenedor.
posiciones_por_contenedor_dia	(id_contenedor, dia) · fecha_hora DESC, id_sensor ASC, id_lectura ASC	Recorrido GPS ordenado, con latitud y longitud.
ultima_lectura_por_sensor	id_sensor	Estado actual, última actividad y detección de inactividad.
ultima_posicion_por_contenedor	id_contenedor	Punto GPS más reciente para mapa/seguimiento.

Cambios de implementación (no necesariamente nuevas tablas)

- **C01 · Ingesta** Validar y normalizar cada lectura, preservar id_lectura y el instante original; escribir historial por sensor, contenedor y GPS cuando aplique.
- **C02 · Ventanas de consulta** Para un intervalo de varios días, consultar las particiones diarias y fusionar/ordenar en la API. No usar ALLOW FILTERING como solución general.
- **C03 · Último valor correcto** Evitar que una lectura tardía sobrescriba una más reciente: comparar timestamps en el proceso de actualización y probar entradas fuera de orden.
- **C04 · Reprocesamiento** Reintentar sin duplicaciones semánticas; si no hay ubicación GPS válida no actualizar la última posición.
- **C05 · Retención** Decidir TTL solo después de acordar qué ventana histórica necesitan los reportes, las anomalías y la demostración.

> **Caso de aceptación imprescindible:** consultar un sensor y un contenedor en un rango que atraviesa dos fechas, mostrar las lecturas ordenadas y reconstruir posiciones GPS sobre un mapa.

Modelo documental a completar

La elección de MongoDB sí está justificada en la primera entrega; el detalle físico no fue adjuntado.

Colección propuesta	Campos de referencia (mínimos)	Uso
eventos_logisticos	id_evento, id_envio, id_contenedor?, tipo, fecha, payload, origen	Incidencias operativas heterogéneas: aduana, demora, desvío.
alertas	id_alerta, id_regla, id_sensor?, id_envio?, valor_observado, estado, fecha, severidad	Resultado de evaluar reglas; mantener estado y confirmación.
conversaciones	id_conversacion, participantes[{tipo,id}], asunto, fecha_creacion, estado	Contexto de intercambios entre partes.
mensajes	id_mensaje, id_conversacion, emisor, contenido, fecha_envio, estado	Historial consultable de mensajería.
reportes_generados	id_reporte, id_solicitud, tipo, parametros, resultado/ubicacion, generado_en	Datos variables del reporte; cobro controlado desde PostgreSQL.
reclamos	id_reclamo, id_cliente, id_envio?, motivo, estado, eventos[]	Previsto en la primera entrega; secundario frente a los 40 requisitos.

Reglas documentales

- **IDs consistentes** Almacenar las referencias con el UUID del registro en PostgreSQL (como String acordada) y validar pertenencia desde la API.
- **Índices mínimos** Eventos por (id_envio, fecha), alertas por (id_sensor, fecha), mensajes por (id_conversacion, fecha_envio).
- **Límites del agregado** No embutir mensajes indefinidamente dentro de una conversación. Mantener referencias para paginación y auditoría.
- **Seguridad** Validar participación en conversaciones y acceso a reportes antes de consultar MongoDB.
- **Trazabilidad cruzada** Persistir id_evento y el vínculo al incidente en PostgreSQL cuando un evento derive en incidente contable/operativo.

> **Decisión de frontera:** las reglas configurables, solicitudes y cargos facturables viven en PostgreSQL; los eventos y resultados de estructura variable pueden quedar en MongoDB. Evitar dos fuentes de verdad para el mismo saldo o estado crítico.

Correcciones de coherencia

El documento actual declara ocho tipos de nodos; el principal ajuste es alinear las relaciones con el DER relacional.

N01 Unificar identificadores

PostgreSQL usa UUID para Cliente, Envio, Tramo, Operador e Incidente; Neo4j usa Integer en esas etiquetas. Utilizar el UUID lógico como String en el grafo.

N02 Arreglar envío-contenedor

La relación ASIGNADO_A aparece 1:1, mientras PostgreSQL permite múltiples vínculos con fechas. Admitir N:M temporal si el negocio lo requiere y guardar vigencia.

N03 Conectar incidentes

Agregar :Envio-[:REGISTRA]->:Incidente; agregar :Tramo-[:REGISTRA]->:Incidente cuando se conozca el tramo, evitando depender solo de una relación.

N04 Estabilizar ubicaciones

No usar solamente nombre_puerto_nodo como identidad global. Incorporar id_ubicacion/código estable y país.

N05 Sincronizar e indexar

Crear nodos/relaciones mediante MERGE y consumo idempotente. Conservar constraints UNIQUE e índices y verificar SHOW CONSTRAINTS/INDEXES.

Consultas demostrables

- **Camino de negocio** Dado un cliente: encontrar envíos, contenedores, tramos, vehículo y operador asociado.
- **Recorrido lógico** Dado un envío: obtener tramos ordenados, sus ubicaciones de origen y destino e incidentes asociados.
- **Consistencia** Tomar un id_envio de PostgreSQL y comprobar que el mismo UUID se encuentra una sola vez en Neo4j.

IoT, monitoreo y alertas

Estos flujos convierten los modelos de datos en capacidades visibles y verificables.

F01 | Ingesta IoT

Sensor / simulador envía lectura con id único y fecha → API valida sensor y asignación vigente → Cassandra guarda histórico → proyección de último estado / GPS → acuse de recepción.

F02 | Evaluación de reglas

Nueva lectura o tarea periódica → PostgreSQL carga reglas activas → compara umbrales/inactividad → MongoDB registra alerta → PostgreSQL registra incidente si corresponde.

F03 | Seguimiento GPS

API recibe id_contenedor + intervalo → consulta particiones de Cassandra → depura coordenadas inválidas → ordena temporalmente → envía puntos al componente de mapa.

F04 | Relaciones en grafo

Alta/cambio de envío o tramo en PostgreSQL → evento sincroniza Neo4j → consulta Cypher arma red cliente-envío-tramo-vehículo-operador.

Excepciones que el equipo debe probar

- **Lectura atrasada** Una lectura antigua no puede reemplazar la última lectura vigente.
- **Sensor sin GPS** Se registra telemetría sin inventar latitud ni longitud.
- **Alerta repetida** Las lecturas equivalentes no disparan alertas duplicadas indefinidamente; definir deduplicación y reapertura.
- **Destino temporalmente caído** Si MongoDB o Neo4j falla, el proceso de sincronización permite recuperación sin pérdida de datos.

Finanzas, reportes y predicción

El diseño físico debe complementarse con servicios de negocio y reglas reproducibles.

F05 | Facturación automática

Hito de entrega / consumo de reporte → leer tarifa vigente → calcular detalle → emitir factura → crear movimiento de cuenta, todo en transacción relacional con clave idempotente.

F06 | Cobros y pagos

Alta de pago → validar tercero, monto y factura / imputación → registrar pago y movimiento → recalcular/conciliar saldo de forma transaccional.

F07 | Cierre mensual

Seleccionar cuenta y período → validar movimientos → calcular saldo inicial, débitos/créditos y saldo final → guardar cierre único → bloquear ajustes no autorizados.

F08 | Reportes facturables

Solicitar reporte parametrizado → validar permisos → registrar solicitud PG → ejecutar consulta → guardar resultado → tarifar y facturar → entregar reporte.

F09 | Predicción y riesgos

Construir histórico de entregas y eventos → producir ETA/riesgo con baseline estadístico o modelo simple → validar con casos históricos → registrar resultado y explicar variables.

F10 | Mensajería

Crear conversación entre participantes autorizados → persistir mensaje/fecha/estado en MongoDB → consultar por conversación → validar aislamiento por cliente.

> **Predicción ≠ consulta de atrasados:** una predicción debe anticipar un resultado futuro. Para la entrega, un baseline por ruta/medio/transporte con evaluación histórica es defendible si se explicitan sus límites y se muestran ejemplos reproducibles.

Requisitos 01 a 08

Cada fila tiene una acción de implementación y una demostración concreta. Ninguna se da por aprobada sin ejecución real.

N°	Punto exigido	Qué desarrollar	Prueba de aceptación
01	ABM clientes	CRUD + validaciones de CUIT y unicidad.	Alta, edición, listado y baja lógica; duplicado rechazado.
02	ABM usuarios y roles	CRUD + asignación y comprobación de permisos.	Usuario con rol distinto recibe acceso distinto.
03	ABM sesiones activas	Expiración, revocación y consulta de sesión vigente.	Sesión expirada/revocada no accede; activa sí.
04	ABM proveedores	CRUD con estados y unicidad de CUIT.	Alta y consulta desde un tramo o factura.
05	ABM operadores logísticos	CRUD de operadores, vehículos y relaciones.	Operador visible en tramos, facturación y grafo.
06	ABM envíos, contenedores y sensores	CRUD y navegación relacional completa.	Un envío muestra contenedores y sus sensores.
07	ABM contenedores de un envío	Asociar/desasociar con vigencia histórica.	Asociación activa e histórica verificables.
08	ABM sensores de un contenedor	Asociar/desasociar, controlar pertenencia temporal.	Sensores correctos por contenedor y por fecha.

Para marcar como completo: modelo creado + datos de prueba + proceso implementado + resultado visible + evidencia guardada. Las cuatro condiciones son necesarias; un endpoint que responde sin datos no basta.

Evidencia a guardar por requisito

- Request/consulta ejecutada con parámetros concretos.
- Respuesta y datos de origen que permiten verificar el resultado.
- Prueba de error o caso límite cuando aplique (duplicados, fechas, permisos).

Requisitos 09 a 16

Cada fila tiene una acción de implementación y una demostración concreta. Ninguna se da por aprobada sin ejecución real.

N°	Punto exigido	Qué desarrollar	Prueba de aceptación
09	ABM reglas y alertas	regla_alerta en PG + alerta ejecutada en Mongo.	Nueva regla supera umbral y genera alerta.
10	Facturación y cobros de clientes	Factura + detalle + pago + movimiento/conciliación.	Cobro parcial/total modifica saldo sin duplicar.
11	Solicitud de reportes	Solicitud parametrizada con estado y resultado.	Usuario solicita y descarga/visualiza reporte.
12	Facturación automática	Tarifa + servicio que crea factura según hito.	Dos ejecuciones del lote generan una sola factura.
13	Registro de pagos	CRUD pago e imputación verificable.	Pago figura en factura, cuenta y movimientos.
14	Gestión de cuentas corrientes	Débitos, créditos, saldo y trazabilidad.	Saldo coincide con suma de movimientos.
15	Cierre mensual	cierre_cuenta + consolidación + bloqueo.	Cierre por cuenta/mes reproduce el saldo; no duplica.
16	Facturación y pagos a terceros	Distinguir proveedores/operadores y egresos.	Factura y pago a cada tercero reflejan su cuenta.

> **Para marcar como completo:** modelo creado + datos de prueba + proceso implementado + resultado visible + evidencia guardada. Las cuatro condiciones son necesarias; un endpoint que responde sin datos no basta.

Evidencia a guardar por requisito

- Request/consulta ejecutada con parámetros concretos.
- Respuesta y datos de origen que permiten verificar el resultado.
- Prueba de error o caso límite cuando aplique (duplicados, fechas, permisos).

Requisitos 17 a 24

Cada fila tiene una acción de implementación y una demostración concreta. Ninguna se da por aprobada sin ejecución real.

N°	Punto exigido	Qué desarrollar	Prueba de aceptación
17	Monitoreo de envíos, sensores y eventos	Vista integrada de PG/Cassandra/Mongo.	Se ve envío con lectura y eventos asociados.
18	Registro continuo de IoT	Ingesta y escritura en tablas desnormalizadas.	Serie de lecturas consecutivas consultable.
19	Histórico por contenedor y sensor	Consultas por rango y unión de días.	Histórico de ambos con intervalo de dos días.
20	Reconstrucción del recorrido	Ordenar posiciones GPS y mapear secuencia.	Mapa dibuja itinerario con puntos en tiempo.
21	Visualización GPS	Última y múltiples posiciones válidas.	Mapa presenta ubicación y trayectoria.
22	Control de estado de sensores	Estado por última lectura, batería e inactividad.	Sensor sin lecturas queda marcado inactivo.
23	Alertas de estado de sensores	Umbrales de batería y sin conexión.	Alerta visible con condición y timestamp.
24	Gestión de trayectos	Orden, vehículo, operador, fechas y estados.	Un envío devuelve tramos en secuencia correcta.

Para marcar como completo: modelo creado + datos de prueba + proceso implementado + resultado visible + evidencia guardada. Las cuatro condiciones son necesarias; un endpoint que responde sin datos no basta.

Evidencia a guardar por requisito

- Request/consulta ejecutada con parámetros concretos.
- Respuesta y datos de origen que permiten verificar el resultado.
- Prueba de error o caso límite cuando aplique (duplicados, fechas, permisos).

Requisitos 25 a 32

Cada fila tiene una acción de implementación y una demostración concreta. Ninguna se da por aprobada sin ejecución real.

N°	Punto exigido	Qué desarrollar	Prueba de aceptación
25	Detección de incidentes	Reglas/eventos crean incidente con tramo opcional.	Evento de prueba crea incidente consultable.
26	Generación de alertas	Evaluar, persistir, consultar y cerrar alertas.	Regla activa produce alerta que cambia de estado.
27	Ranking de rutas con incidentes	Agrupar incidentes por ruta/tramo definido.	Ranking ordenado con recuentos comprobables.
28	Ranking de clientes por volumen	SUM(volumen_m3) con criterio de envíos válido.	Orden descendente coincide con dataset preparado.
29	Predicción de atrasos	ETA/riesgo futuro sobre históricos + validación.	Pronóstico para envío aún no entregado.
30	Red clientes-operadores-envíos	Grafo sincronizado con ids compartidos.	Cypher obtiene red de un cliente concreto.
31	Análisis histórico de desempeño	Definir KPI por período y entidades.	Comparar dos períodos con resultados distintos.
32	Anomalías de temperatura	Umbral/salida anómala en lecturas históricas.	Lectura fuera de rango produce detección.

Para marcar como completo: modelo creado + datos de prueba + proceso implementado + resultado visible + evidencia guardada. Las cuatro condiciones son necesarias; un endpoint que responde sin datos no basta.

Evidencia a guardar por requisito

- Request/consulta ejecutada con parámetros concretos.
- Respuesta y datos de origen que permiten verificar el resultado.
- Prueba de error o caso límite cuando aplique (duplicados, fechas, permisos).

Requisitos 33 a 40

Cada fila tiene una acción de implementación y una demostración concreta. Ninguna se da por aprobada sin ejecución real.

N°	Punto exigido	Qué desarrollar	Prueba de aceptación
33	Reportes parametrizados facturables	Parámetros, resultado, tarifa y cargo.	Reporte con filtros genera precio/factura vinculada.
34	Reportes predictivos	Guardar predicción, riesgo y explicación.	Reporte incluye ETA/score para envíos actuales.
35	Reporte de envíos con filtros	País, estado, cliente, demorados, incidente abierto.	Cada filtro y combinación produce salida correcta.
36	Máx./mín. de duración por cliente	Usar fechas reales y excluir casos sin entrega.	Cliente tiene min/max consistentes con dataset.
37	Promedio de incidentes por dimensiones	Definir país, estado y denominador explícito.	Promedio por país/estado/cliente reproducible.
38	Tiempo promedio de entrega	Promediar duración real en envíos finalizados.	Media manual coincide con reporte.
39	Costos por transporte	Agrupar costo_real de tramos por medio.	Totales por aéreo/marítimo/terrestre coinciden.
40	Mensajería con las partes	Conversaciones y mensajes con permisos.	Cliente/proveedor/operador envían y consultan; aislamiento.

Para marcar como completo: modelo creado + datos de prueba + proceso implementado + resultado visible + evidencia guardada. Las cuatro condiciones son necesarias; un endpoint que responde sin datos no basta.

Evidencia a guardar por requisito

- Request/consulta ejecutada con parámetros concretos.
- Respuesta y datos de origen que permiten verificar el resultado.
- Prueba de error o caso límite cuando aplique (duplicados, fechas, permisos).

Roadmap de implementación

Orden por dependencias: todos los requisitos son obligatorios; la prioridad indica secuencia de desarrollo, no exclusión.

SPRINT 0

Contrato de datos y esquema

Acordar IDs, estados y KPIs; corregir PostgreSQL/Neo4j; documentar modelo Mongo; validar CQL.

ROL SUGERIDO: DATOS / ARQUITECTURA

SPRINT 1

ABM y operaciones básicas

Clientes, usuarios, roles, proveedores, operadores, envíos, contenedores, sensores y sesiones.

ROL SUGERIDO: BACKEND / UI

SPRINT 2

Ingesta y seguimiento

Simulador IoT; Cassandra; tablero de estado; GPS; rutas; eventos y alertas.

ROL SUGERIDO: IOT / BACKEND / UI

SPRINT 3

Finanzas y mensajería

Tarifas, detalles, pagos, movimientos, cierres, facturación automática y conversaciones.

ROL SUGERIDO: BACKEND / FINANZAS

SPRINT 4

Analítica y predicción

Rankings, reportes con filtros, reportes facturables, KPIs, baseline predictivo y riesgos.

ROL SUGERIDO: DATOS / ANALÍTICA

SPRINT 5

Pruebas de promoción

Dataset, pruebas de los 40 casos, evidencias, demos y correcciones.

ROL SUGERIDO: QA / TODO EQUIPO

Distribución recomendada: separar responsables de datos, backend, interfaz, analítica y QA, pero asignar un dueño por requisito. Los sprints son bloques lógicos y no plazos garantizados.

Dataset de demostración

Sembrar datos intencionalmente diversos para que todos los rankings, reportes, alertas y predicciones devuelvan resultados.

Conjunto	Volumen de referencia propuesto (no exigido por el profesor)
Maestros	8 clientes · 5 operadores · 8 proveedores · usuarios con distintos roles
Logística	80 envíos sintéticos · varios países · 3 medios de transporte · múltiples tramos
IoT	25 contenedores · 40 sensores · varios días de telemetría, incluso GPS y lecturas tardías
Incidentes	20 o más eventos variados · estados abiertos/cerrados · al menos 2 rutas con casos
Finanzas	Facturas emitidas/pagadas/parciales · terceros · 3 períodos · cierres y tarifas
Analítica	Envíos entregados puntuales/demorados y en tránsito para probar baselines
Mensajería	Conversaciones con clientes, proveedores y operadores, más un intento denegado

Casos adversos obligatorios

- **Duplicados** Crear dos veces la misma factura automática, alerta o lectura; verificar comportamiento idempotente.
- **Fechas y ventanas** Cruzar medianoche y cierre de mes; verificar particiones Cassandra, estados temporales y saldo anterior.
- **Errores de autorización** Intentar acceder a envíos, reportes o mensajes de otra organización.
- **Datos incompletos** Sensor sin GPS, tramo sin llegada real, incidente sin tramo y envío no entregado.
- **Falla parcial** Reiniciar consumidor de eventos tras un error y demostrar sincronización sin duplicados.
- **Análisis** Comprobar las consultas analíticas contra cálculos manuales de un subconjunto pequeño.

> **Requisito de demo:** los números anteriores son una propuesta de dataset sintético, no mínimos oficiales. Ajustarlos al tiempo del equipo, manteniendo suficiente diversidad para que los 40 puntos se puedan mostrar.

Definición de “terminado”

La promoción depende de demostrar capacidad operativa, no solo de exhibir los archivos de diseño.

Checklist antes de presentar

- [ ] **Diseño aprobado**
  DER SQL, CQL, MongoDB y Neo4j actualizados y consistentes.
- [ ] **Esquema reproducible**
  Scripts de creación, migración y carga ejecutables desde cero.
- [ ] **40 flujos operativos**
  Cada punto dispone de una acción real y respuesta no vacía.
- [ ] **Dataset versionado**
  Los datos sintéticos reproducen todos los casos de prueba.
- [ ] **Resultados comprobables**
  Rankings, cálculos y reportes contrastados con valores esperados.
- [ ] **Seguridad mínima**
  Autorización por rol/cliente, no exposición de información ajena.
- [ ] **Resiliencia demostrada**
  Pruebas de idempotencia, sincronización y registros fallidos.
- [ ] **Carpeta de evidencias**
  40 subcarpetas R01–R40 con request, respuesta, consulta y captura.
- [ ] **Guion de demo**
  Recorrido continuo: envío → IoT/GPS → alerta/incidente → reporte → factura/cierre.

Fuentes y límites de la auditoría

Documentos revisados: “Condiciones necesarias para Promocionar.docx” (40 puntos); “Ing. Datos II Primer Entrega Grupo 4 (1).pdf” (estrategia políglota); “DER-PostgreeSQL-TP_Ing_Datos2 (1).webp”; “ModeladoCassandraGrupo4 (1).drawio”; “Modelo_Fisico_Grafos_Neo4j.docx”.

Límite: no se aportaron código backend/frontend, migraciones SQL/CQL, esquemas MongoDB, consultas ejecutadas ni resultados de pruebas. Las medidas y entidades nuevas de este documento son recomendaciones de diseño, no componentes constatados.